

SINTAXIS, PROPIEDADES Y MÉTODOS DE APOYO

Propiedades del objeto event	
Propiedades	Descripción
bubbles	Valor booleano que indica si el evento burbujea o no.
cancelable	Valor booleano que indica si el evento se puede cancelar.
currentTarget	El elemento al que se asignó el evento. Por ejemplo si tenemos un evento de click en un <code>divA</code> que contiene un hijo <code>divB</code> . Si hacemos click en <code>divB</code> , <code>currentTarget</code> referenciará a <code>divA</code> (el elemento dónde se asignó el evento) mientras que <code>target</code> devolverá <code>divB</code> , el elemento dónde ocurrió el evento.
defaultPrevented	Valor booleano que indica cuando <i>true</i> , que <code>preventDefault()</code> ha sido llamado.
eventPhase	Un valor integer que indica la fase del evento que está siendo procesada. Fase de captura (1), en destino (2) o fase de burbujeo (3).
target	El elemento destino del evento.
type	String que contiene el nombre del evento que fue lanzado.

Métodos del objeto event	
Métodos	Descripción
preventDefault()	Cancela cualquier acción asociada por defecto a un evento.
stopImmediatePropagation()	Cancela cualquier evento captura o burbujeo y previene que cualquier otro manejador de evento sea llamado.
stopPropagation()	Evita que un evento burbujee. Por ejemplo si tenemos un <code>divA</code> que contiene un <code>divB</code> hijo. Cuando asignamos un evento de click a <code>divA</code>, si hacemos click en <code>divB</code>, por defecto se dispararía también el evento en <code>divA</code> en la fase de burbujeo. Para evitar esto se puede llamar a <code>stopPropagation()</code> en <code>divB</code>. Para ello creamos un evento de click en <code>divB</code> y le hacemos <code>stopPropagation()</code>.

Propiedades de los elementos de un formulario	
Propiedad	Descripción
form	Devuelve una referencia al formulario al que pertenece el elemento. Por ejemplo: <code>document.getElementById("id_elemento").form</code>
name	Devuelve el valor del atributo name.
type	Devuelve el tipo de elemento. Para elementos <code><input></code> (text, checkbox, ...) , su valor coincide con el atributo <i>type</i> . Para las listas desplegables <code><select></code> de selección única, su valor es <i>select-one</i> , mientras que para las de selección múltiple, será <i>select-multiple</i> . En el caso de <code><textarea></code> , es <i>textarea</i> .
value	Devuelve y permite modificar el valor de un elemento. En los campos de texto se obtiene el texto escrito.

Los eventos más utilizados en la gestión de formularios, son:

Eventos más utilizados en un formulario	
Evento	Descripción
click	Se produce cuando se hace clic sobre un elemento. Muy utilizado en los tipos de botones que permite definir HTML.
change	Se produce cuando hay un cambio en el contenido de un campo de texto o lista desplegable.
focus	Se produce cuando se selecciona o recibe el foco algún elemento de la página.
blur	Se produce cuando un elemento pierde el foco.

Propiedades del objeto form.	
Propiedad	Descripción
acceptCharset	Devuelve o modifica el conjunto de caracteres utilizado para codificar los datos del formulario al ser enviado al servidor.
action	Devuelve o modifica la URL o la ubicación a la que se enviarán los datos del formulario.
enctype	Devuelve o modifica el tipo de codificación de datos utilizada para enviar los datos del formulario al servidor.
length	Devuelve el número de elementos de un formulario.
method	Devuelve o modifica el método HTTP utilizado para enviar los datos del formulario al servidor (por ejemplo "GET" o "POST").
name	Devuelve o modifica el valor del atributo <i>name</i> en un formulario.
target	Devuelve o modifica el nombre o la ventana de destino donde se mostrará la respuesta del servidor después de enviar el formulario

Propiedades del objeto form.	
Propiedad	Descripción
	(<i>_black</i> , <i>_self</i> , <i>_parent</i> , <i>_top</i>).

Métodos del objeto form.	
Propiedad	Descripción
<code>reset()</code>	Reinicia un formulario.
<code>submit()</code>	Envía un formulario.

Propiedades del objeto Input de tipo texto.	
Propiedad	Descripción
<code>defaultValue</code>	Devuelve o modifica el valor por defecto del elemento input.
<code>form</code>	Devuelve la referencia al formulario a que pertenece el elemento input.
<code>maxLength</code>	Devuelve o cambia la longitud máxima permitida de caracteres en el elemento input.
<code>name</code>	Devuelve o cambia el valor del atributo <i>name</i> .
<code>readOnly</code>	Devuelve o cambia la propiedad de solo lectura de un elemento input.
<code>size</code>	Devuelve o cambia el tamaño de un elemento input en caracteres.
<code>type</code>	Devuelve el tipo de un elemento input.
<code>value</code>	Devuelve o modifica el contenido del elemento input.

Valores más comunes de la propiedad type de un elemento <input>.	
Valor	Descripción
<code>text</code>	Añade un campo de texto de una sola línea.
<code>password</code>	Añade un campo de contraseña que oculta los caracteres conformen se escriben.
<code>checkbox</code>	Añade una casilla de verificación para seleccionar una o varias opciones.
<code>radio</code>	Añade un botón de opción para seleccionar una única opción de un grupo.
<code>submit</code>	Añade un botón de envío de formulario.
<code>reset</code>	Añade un botón de reinicio que resetea los valores por defecto, el formulario.
<code>button</code>	Añade un botón genérico al que se le asignará funcionalidad a través de un manejador de evento.
<code>file</code>	Añade un campo de carga de archivo, para seleccionar de los archivos locales.
<code>email</code>	Añade un campo que valida si el formato del correo electrónico es correcto.
<code>number</code>	Añade un campo de entrada numérica, que acepta solo valores numéricos.
<code>date</code>	Añade un campo de entrada para seleccionar una fecha específica.
<code>color</code>	Añade un campo de selección de color.

Métodos del objeto Input de tipo texto.	
Propiedad	Descripción
<code>select()</code>	Selecciona el contenido de un elemento input.

Propiedades del objeto Input de tipo checkbox.	
Propiedad	Descripción
<code>checked</code>	Devuelve o modifica el estado de selección de un checkbox.
<code>defaultChecked</code>	Devuelve el valor por defecto del atributo checked.
<code>form</code>	Devuelve la referencia al formulario que contiene el checkbox.
<code>name</code>	Devuelve o modifica el nombre de un checkbox.
<code>type</code>	Devuelve el tipo de elemento de formulario.
<code>value</code>	Devuelve o modifica el valor del atributo value.

Propiedades del objeto RegExp	
Propiedad	Descripción
<code>global</code>	Devuelve <code>true</code> si se ha utilizado el modificador "g" en la expresión regular.
<code>ignoreCase</code>	Devuelve <code>true</code> si se ha utilizado el modificador "i" en la expresión regular.
<code>lastIndex</code>	Devuelve el índice donde comenzar la siguiente búsqueda.
<code>multiline</code>	Devuelve <code>true</code> si se ha utilizado el modificador "m" en la expresión regular.
<code>source</code>	Devuelve el texto de la expresión regular (sin los flags).
<code>flags</code>	Devuelve los flags que están activos en la expresión regular.

Métodos del objeto RegExp	
Método	Descripción
<code>toString()</code>	Devuelve la representación literal de la expresión regular.
<code>exec()</code>	Busca la coincidencia en una cadena. Devolverá un array con todas las coincidencias de la expresión regular en el texto.
<code>test()</code>	Busca la coincidencia en una cadena. Devolverá <code>true</code> o <code>false</code> .

Caracteres especiales utilizados en Expresiones Regulares			
Carácter	Coincidencias	Patrón	Ejemplo de cadena
<code>^</code>	Al inicio de una cadena	<code>/^Esto/</code>	Coincidencia en "Esto es..."
<code>\$</code>	Al final de la cadena	<code>/final\$/</code>	Coincidencia en "Esto es el final".
<code>*</code>	Coincide 0 o más veces	<code>/se*/</code>	Que la "e" aparezca 0 o más veces: "seeee" y también "se".
<code>?</code>	Coincide 0 o 1 vez	<code>/ap?/</code>	Que la p aparezca 0 o 1 vez: "apple"

Caracteres especiales utilizados en Expresiones Regulares			
Carácter	Coincidencias	Patrón	Ejemplo de cadena
			y "and".
+	Coincide 1 o más veces	/ap+/	Que la "p" aparezca 1 o más veces: "apple" pero no "and".
{n}	Coincide exactamente n veces	/ap{2}/	Que la "p" aparezca exactamente 2 veces: "apple" pero no "apabullante".
{n, }	Coincide n o más veces	/ap{2, }/	Que la "p" aparezca 2 o más veces: "apple" y "appple" pero no en "apabullante".
{n, m}	Coincide al menos n, y máximo m veces	/ap{2, 4}/	Que la "p" aparezca al menos 2 veces y como máximo 4 veces: "apppppple" (encontrará 4 "p").
.	Cualquier carácter excepto nueva línea	/a.e/	Que aparezca cualquier carácter, excepto nueva línea entre la a y la e: "ape" y "axe".
[...]	Cualquier carácter entre corchetes	/a[px]e/	Que aparezca alguno de los caracteres "p" o "x" entre la a y la e: "ape", "axe", pero no "ale".
[^...]	Cualquier carácter excepto los que están entre corchetes	/a[^px]/	Que aparezca cualquier carácter excepto la "p" o la "x" después de la letra a: "ale", pero no "axe" o "ape".
\b	Coincide con el inicio de una palabra	/\bno/	Que "no" esté al comienzo de una palabra: "novedad".
\B	Coincide al final de una palabra	/\Bno/	Que "no" esté al final de una palabra: "este invierno" ("no" de "invierno").
\d	Dígitos del 0 al 9	/\d{3}/	Que aparezcan exactamente 3 dígitos: "Ahora en 456".
\D	Cualquier carácter que no sea un dígito	/\D{2, 4}/	Que aparezcan mínimo 2 y máximo 4 caracteres que no sean dígitos: encontrará la cadena "Ahor" en "Ahora en 456".
\w	Coincide con caracteres del tipo (letras, dígitos, subrayados)	/\w/	Que aparezca un carácter (letra, dígito o subrayado): "J" en "JavaScript".
\W	Coincide con caracteres que no sean (letras, dígitos, subrayados)	/\W/	Que aparezca un carácter (que no sea letra, dígito o subrayado): "%" en "100%".
\n	Coincide con una nueva línea		
\s	Coincide con un espacio en blanco		
\S	Coincide con un carácter que no es un espacio en blanco		

Caracteres especiales utilizados en Expresiones Regulares			
Carácter	Coincidencias	Patrón	Ejemplo de cadena
\t	Un tabulador		
(x)	Capturando paréntesis		Recuerda los caracteres.
\r	Un retorno de carro		
?=n	Cualquier cadena que está seguida por la cadena n indicada después del igual.	/la(?:= mundo)	Hola mundo mundial.

Creación de una cookie

Para crear una cookie se utiliza la propiedad document.cookie. La estructura es la siguiente:

```
document.cookie = "nombre=valor; expires=fecha; path=ruta; domain=dominio; secure";
```

ARRAY

Propiedades del objeto Array	
Propiedad	Descripción
constructor	Devuelve la función que creó el prototipo del objeto array.
length	Ajusta o devuelve el número de elementos en un array.
prototype	Te permite añadir propiedades y métodos a un objeto.

Métodos del objeto Array	
Métodos	Descripción
at(pos)	Devuelve el elemento de la posición <i>pos</i> . Admite pasar valores negativos, que implica empezar a contar desde el final del array hasta el principio.
concat(elem1, elem2,...)	Une dos o más arrays o elementos, al final del array, y devuelve una copia con los arrays o elementos unidos. No modifica el array original.
copyWithin(pos, inicio, fin)	Modifica el array en la posición <i>pos</i> , copiando los elementos desde <i>inicio</i> a <i>fin</i> .
every()	Ejecuta una función sobre cada elemento del

Métodos del objeto Array	
Métodos	Descripción
	array, devolviendo <code>true</code> si la función devuelve <code>true</code> para cada elemento.
<code>filter()</code>	Ejecuta una función sobre cada elemento del array, devolviendo un array con todos los elementos para los cuales la función devuelve <code>true</code> .
<code>fill(elementos, inicio, fin)</code>	Modifica el array rellenando <i>elementos</i> desde <i>inicio</i> hasta <i>fin</i> .
<code>find(función(elem))</code>	Devuelve el primer elemento del array que cumple la condición definida por la función. Devuelve -1 si no existe elemento que cumpla la condición.
<code>findIndex(función(elem))</code>	Devuelve el primer índice del elemento que cumple la condición definida en la función.
<code>findLast(función(elem))</code>	Devuelve el último elemento del array que cumple la condición definida por la función. Devuelve -1 si no existe elemento que cumpla la condición.
<code>findLastIndex(función(elem))</code>	Devuelve el último índice del elemento que cumple la condición definida en la función.
<code>forEach()</code>	Ejecuta una función sobre cada elemento del array, y no tiene valor de retorno (se comporta como el bucle <code>for</code> sobre un array).
<code>includes(elem)</code>	Devuelve <i>true</i> o <i>false</i> si el elemento existe o no dentro del array.
<code>includes(elem, desde)</code>	Devuelve <i>true</i> si el elemento existe dentro del array a partir de la posición <i>desde</i> .
<code>indexOf(elem)</code>	Devuelve la posición de la primera aparición del elemento buscado. La búsqueda comienza desde el comienzo del array hasta el final. Si no existe, devuelve -1.
<code>indexOf(elem, desde)</code>	Igual que <code>indexOf(elem)</code> pero buscando a partir de la posición <i>desde</i> .
<code>join(separador)</code>	Une todos los elementos de un array en una cadena de texto, separando cada elemento con el <i>separador</i> .
<code>lastIndexOf(elem)</code>	Devuelve la posición de la primera aparición del elemento buscado. La búsqueda comienza desde el final del array hasta el comienzo. Si no existe, devuelve -1.
<code>lastIndexOf(elem, desde)</code>	Devuelve la posición del elemento buscando desde la posición <i>desde</i> hasta el inicio del array. Si no existe, devuelve -1.
<code>map()</code>	Ejecuta una función sobre cada elemento del array, devolviendo el resultado de cada

Métodos del objeto Array	
Métodos	Descripción
	llamada de la función en un array.
<code>pop()</code>	Elimina el último elemento de un array y devuelve ese elemento.
<code>push(elem1, elem2,...)</code>	Añade nuevos elementos al final de un array, y devuelve la nueva longitud.
<code>reduce(función(accumulator, elemento))</code>	Ejecuta una función sobre todos los elementos del array, construyendo un valor final que será devuelto. Comienza desde el inicio del array hasta el final.
<code>reduceRight()</code>	Ejecuta una función sobre todos los elementos del array, construyendo un valor final que será devuelto. Comienza desde el final del array hasta el inicio.
<code>reduceRight(función(accumulator, elemento))</code>	Ejecuta una función sobre todos los elementos del array, construyendo un valor final que será devuelto. Comienza desde el final del array hasta el inicio.
<code>reverse()</code>	Invierte el orden de los elementos en un array.
<code>shift()</code>	Devuelve el primer elemento del array, lo elimina y actualiza su longitud.
<code>slice(inicio, fin)</code>	Selecciona una parte de un array y devuelve el nuevo array. Excluyendo el índice <i>fin</i> .
<code>some()</code>	Ejecuta una función sobre cada elemento del array, devolviendo <code>true</code> si la función devuelve <code>true</code> para al menos un elemento.
<code>sort(criterio)</code>	Ordena los elementos de un array según el criterio especificado en la función <i>criterio</i> .
<code>splice(inicio, cantidad)</code>	Modifica el array eliminando <i>cantidad</i> elementos desde la posición <i>inicio</i> .
<code>splice(inicio, cantidad, elem1, elem2, ...)</code>	Elimina del array <i>cantidad</i> de elementos desde la posición <i>inicio</i> , y en esa misma posición introduce los elementos <i>elem1</i> , <i>elem2</i> ,
<code>split(separador)</code>	Devuelve un array a partir de una cadena, dividiendo dicha cadena en elementos delimitados por <i>separador</i> .
<code>toString()</code>	Convierte un array a una cadena y devuelve el resultado.
<code>unshift(elem1, elem2,...)</code>	Añade nuevos elementos al comienzo de un array, y devuelve la nueva longitud.
<code>valueOf()</code>	Devuelve el valor primitivo de un array.

FUNCIONES

Funciones flecha o arrow functions.

```
(argumentos) => {
    // sentencias a ejecutar
}
```

Propiedades del objeto Function	
Propiedad	Descripción
constructor	Devuelve la función que creó el objeto Function.
length	Devuelve el número de parámetros esperados por la función (los que aparecen en la declaración).
name	Devuelve el nombre de la función tal como se especificó en su creación. Será anonymous o cadena vacía para funciones creadas de forma anónima.
prototype	Te permitirá añadir propiedades y métodos a un objeto.

Métodos del objeto Function	
Método	Descripción
apply(thisObject, arguments)	Llama a la función pasando el valor del objeto this y un array de argumentos.
bind(objeto)	Crea una nueva instancia de función, cuyo valor del objeto this será ligado al que se le pase como parámetro.
call(thisObject, arg1, ...)	Mismo comportamiento que apply() pero los argumentos son pasados individualmente directamente, en lugar de un array.

CLASES

```
export class Clase {
}

```

```
<script type="module">
  import { Clase } from "./Clase.js"
</script>

```

Palabras reservadas más comunes	
Nombre	Uso
var	Utilizada para declarar una variable.
if .. else	Estructura condicional.
for	Estructura de repetición. Se ejecutará mientras la condición sea verdadera,
do ... while	Estructura de repetición. Se ejecutará mientras la condición sea verdadera.
switch	Sentencia condicional.
break	Termina un switch o un bucle.
continue	Sale del bucle y se coloca al comienzo de este.
function	Declara una función.
return	Sale de una función.
try – catch	Utilizadas para el manejo de excepciones.
let	Declaración de variable.
const	Declaración de constantes.

Categorías de operadores en JavaScript	
Tipo	Qué realizan
Comparación.	Comparan los valores de 2 operandos, devolviendo un resultado de true o false (se usan extensivamente en sentencias condicionales como <code>if... else</code> y en instrucciones <code>bucle</code>). <code>== != === !== >>= <<=</code>
Aritméticos.	Unen dos operandos para producir un único valor que es el resultado de una operación aritmética u otra operación sobre ambos operandos. <code>+ - * / % ++ -- +valor -valor</code>
Asignación.	Asigna el valor a la derecha de la expresión a la variable que está a la izquierda. <code>= += -= *= /= %= <<= >= >>= >>>=</code> <code>&= = ^= []</code>
Boolean.	Realizan operaciones booleanas aritméticas sobre uno o dos operandos

	booleanos. && !
Bit a Bit.	Realizan operaciones aritméticas o de desplazamiento de columna en las representaciones binarias de dos operandos. & ^ ~ << >> >>>
Objeto.	Ayudan a los scripts a evaluar la herencia y capacidades de un objeto particular antes de que tengamos que invocar al objeto y sus propiedades o métodos. . [] () delete in instanceof new this
Misceláneos.	Operadores que tienen un comportamiento especial. , ?: typeof void

Operadores de comparación en JavaScript			
Sintaxis	Nombre	Tipos de operandos	Resultados
==	Igualdad.	Todos.	Boolean.
!=	Distinto.	Todos.	Boolean.
===	Igualdad estricta.	Todos.	Boolean.
!==	Desigualdad estricta.	Todos.	Boolean.
>	Mayor que .	Todos.	Boolean.
>=	Mayor o igual que.	Todos.	Boolean.
<	Menor que.	Todos.	Boolean.
<=	Menor o igual que.	Todos.	Boolean.

Operadores aritméticos en JavaScript			
Sintaxis	Nombre	Tipos de Operando	Resultados
+	Más.	integer, float, string.	integer, float, string.
-	Menos.	integer, float.	integer, float.
*	Multipliación.	integer, float.	integer, float.
/	División.	integer, float.	integer, float.
%	Módulo.	integer, float.	integer, float.
++	Incremento.	integer, float.	integer, float.
--	Decremento.	integer, float.	integer, float.
+valor	Positivo.	integer, float, string.	integer, float.
-valor	Negativo.	integer, float, string.	integer, float.

Operadores de asignación en JavaScript			
Sintaxis	Nombre	Ejemplo	Significado
=	Asignación.	x = y	x = y
+=	Sumar un valor.	x += y	x = x + y
-=	Substraer un valor.	x -= y	x = x - y
*=	Multiplicar un valor.	x *= y	x = x * y
/=	Dividir un valor.	x /= y	x = x / y
%=	Módulo de un valor.	x %= y	x = x % y
<<=	Desplazar bits a la izquierda.	x <<= y	x = x << y
>=	Desplazar bits a la derecha.	x >= y	x = x > y
>>=	Desplazar bits a la derecha rellenando con 0.	x >>= y	x = x >> y
>>>=	Desplazar bits a la derecha.	x >>>= y	x = x >>> y
&=	Operación AND bit a bit.	x &= y	x = x & y
=	Operación OR bit a bit.	x = y	x = x y
^=	Operación XOR bit a bit.	x ^= y	x = x ^ y
[]=	Desestructurando asignaciones.	[a,b]=[c,d]	a=c, b=d

Operadores de boolean en JavaScript			
Sintaxis	Nombre	Operandos	Resultados
&&	AND.	Boolean.	Boolean.
	OR.	Boolean.	Boolean.
!	Not.	Boolean.	Boolean.

Tabla de valores de verdad del operador AND			
Operando Izquierdo	Operador AND	Operando Derecho	Resultado
True	&&	True	True
True	&&	False	False
False	&&	True	False
False	&&	False	False

Tabla de valores de verdad del operador OR			
Operando Izquierdo	Operador OR	Operando Derecho	Resultado
True		True	True
True		False	True
False		True	True
False		False	False

? : (operador condicional)

Este operador condicional es la forma reducida de la expresión `if ... else`.

La sintaxis formal para este operador condicional es:

condicion ? expresión si se cumple la condición: expresión si no se cumple;

Condicionales y bucles.**Construcción `if`**

Sintaxis:

```
if (condición)
{
// instrucciones a ejecutar si se cumple la condición
}
```

Construcción `if ... else`

Sintaxis:

```
if (condición)
{
// instrucciones a ejecutar si se cumple la condición
}
else
{
// instrucciones a ejecutar si no se cumple la condición
}
```

Sentencia `switch`.

Sintaxis:

```
switch (expresion) {
case valor: sentencia
break;
case valor: sentencia
break;
case valor: sentencia
break;
case valor: sentencia
break;
default: sentencia
}
```

Bucle for.

Sintaxis:

```
for (expresión inicial; condición; incremento)
{
// Instrucciones a ejecutar dentro del bucle.
}
```

Bucle while().

Sintaxis:

```
while (condición)
{
// Instrucciones a ejecutar dentro del bucle.
}
```

Bucle do ... while().

Sintaxis:

```
do {
// Instrucciones a ejecutar dentro del bucle.
}while (condición);
```

Propiedades del objeto Number	
Propiedad	Descripción
constructor	Devuelve la función que creó el objeto <code>Number</code> .
MAX_VALUE	Devuelve el número más alto disponible en JavaScript.
MIN_VALUE	Devuelve el número más pequeño disponible en JavaScript.
MAX_SAFE_INTEGER	El valor máximo para realizar cálculos con seguridad de que los resultados son correctos en JavaScript.
MIN_SAFE_INTEGER	El valor mínimo para realizar cálculos con seguridad de que los resultados son correctos en JavaScript.
NaN	La propiedad <i>NaN</i> (<i>Not a Number</i>) indica que el valor de un número no es un número válido. <code>Number.isNaN(NaN)</code> ; sería la forma correcta de comprobarlo.
NEGATIVE_INFINITY	Representa a infinito negativo (se devuelve en caso de overflow).
POSITIVE_INFINITY	Representa a infinito positivo (se devuelve en caso de overflow).
prototype	Permite añadir nuestras propias propiedades y métodos a un objeto.

Métodos del objeto Number	
<code>isFinite()</code>	Comprueba si el número introducido por parámetro es finito.
<code>isInteger()</code>	Comprueba si el número introducido por parámetro es un número entero.
<code>toExponential(x)</code>	Convierte un número a su notación exponencial con la cantidad de decimales expresada por parámetro.
<code>toFixed(x)</code>	Formatea un número con <i>x</i> dígitos decimales después del punto decimal.
<code>toPrecision(x)</code>	Formatea un número a la longitud <i>x</i> , es decir, cantidad total de dígitos que se mostrará.
<code>toString()</code>	Convierte un objeto <code>Number</code> en una cadena. • Si se pone 2 como parámetro se mostrará el número en binario. • Si se pone 8 como parámetro se mostrará el número en octal. • Si se pone 16 como parámetro se mostrará el número en hexadecimal.
<code>valueOf()</code>	Devuelve el valor primitivo de un objeto <code>Number</code> .

Propiedades del objeto Math	
Propiedad	Descripción
<code>E</code>	Devuelve el número Euler (aproximadamente 2.718).
<code>LN10</code>	Devuelve el logaritmo neperiano de 10 (aproximadamente 2.302).
<code>LN2</code>	Devuelve el logaritmo neperiano de 2 (aproximadamente 0.693).
<code>LOG10E</code>	Devuelve el logaritmo base 10 de E (aproximadamente 0.434).
<code>LOG2E</code>	Devuelve el logaritmo base 2 de E (aproximadamente 1.442).
<code>PI</code>	Devuelve el número PI (aproximadamente 3.14159).
<code>SQRT1_2</code>	Devuelve la raíz cuadrada de $\frac{1}{2}$ (aproximadamente 0.707).
<code>SQRT2</code>	Devuelve la raíz cuadrada de 2 (aproximadamente 1.414).

Métodos del objeto Math	
Método	Descripción
<code>abs(x)</code>	Devuelve el valor absoluto de <i>x</i> .
<code>acos(x)</code>	Devuelve el arcocoseno de <i>x</i> , en radianes.
<code>asin(x)</code>	Devuelve el arcoseno de <i>x</i> , en radianes.
<code>atan(x)</code>	Devuelve el arcotangente de <i>x</i> , en radianes con un valor entre $-\pi/2$ y $\pi/2$.
<code>atan2(y, x)</code>	Devuelve el arcotangente del cociente de sus argumentos.
<code>atanh(x)</code>	Devuelve arcotangente hiperbólica de <i>x</i> .

Métodos del objeto Math	
Método	Descripción
<code>cbrt(x)</code>	Devuelve la raíz cúbica de x .
<code>ceil(x)</code>	Devuelve el número x redondeado al alta hacia el siguiente entero.
<code>cos(x)</code>	Devuelve el coseno de x (x está en radianes).
<code>exp(x)</code>	Devuelve el número e (Euler) elevado a x .
<code>expm1(x)</code>	Devuelve el número e (Euler) elevado a x menos 1.
<code>floor(x)</code>	Devuelve el número x redondeado a la baja hacia el anterior entero.
<code>log(x)</code>	Devuelve el logaritmo neperiano (base E) de x .
<code>max(x, y, z, ..., n)</code>	Devuelve el número más alto de los que se pasan como parámetros.
<code>min(x, y, z, ..., n)</code>	Devuelve el número más bajo de los que se pasan como parámetros.
<code>pow(x, y)</code>	Devuelve el resultado de x elevado a y .
<code>random()</code>	Devuelve un número al azar entre 0 y 0,9999... con 16 decimales.
<code>round(x)</code>	Redondea x al entero más próximo.
<code>sign(x)</code>	Devuelve un 1 si x es positivo ó -1 si x es negativo.
<code>sin(x)</code>	Devuelve el seno de x (x está en radianes).
<code>sqrt(x)</code>	Devuelve la raíz cuadrada de x .
<code>tan(x)</code>	Devuelve la tangente de un ángulo.
<code>tanh(x)</code>	Devuelve tangente hiperbólica de x .

Propiedades del objeto Boolean	
<code>constructor</code>	Devuelve la función que creó el objeto <code>Boolean</code> .
<code>prototype</code>	Permitirá añadir propiedades y métodos a un objeto.

Métodos del objeto Boolean	
<code>toString()</code>	Convierte un valor <code>Boolean</code> a una cadena y devuelve el resultado.
<code>valueOf()</code>	Devuelve el valor primitivo de un objeto <code>Boolean</code> .

Date

- `new Date()`: se obtiene la fecha y hora actuales.
- `new Date(cadena)`: convierte un texto con el formato YYYY/MM/DD HH:MM:SS en una fecha.
- `new Date(número)`: el parámetro número representa los milisegundos que han pasado desde el 1/1/1970 a las 00:00:00. Se utiliza para los intervalos de tiempo.
- `new Date(año, mes, día, hora, minuto, segundo, milisegundo)`: crea una fecha en formato UTC a partir de todos los argumentos pasados. No es necesario pasar todos los parámetros, pero si es obligatorio el año y el mes.

- `Date.now()`: devuelve el momento actual representado con un número que será los milisegundos que han pasado desde el 1/1/1970.

Propiedades del objeto Date	
Propiedad	Descripción
constructor	Devuelve la función que creó el objeto <code>Date</code> .
prototype	Te permitirá añadir propiedades y métodos a un objeto.

Métodos del objeto Date	
Métodos	Descripción
<code>getDate()</code>	Devuelve el día del mes (de 1-31).
<code>getDay()</code>	Devuelve el día de la semana (de 0-6). 0 es domingo.
<code>getFullYear()</code>	Devuelve el año en 4 dígitos.
<code>getHours()</code>	Devuelve la hora (de 0-23).
<code>getMilliseconds()</code>	Devuelve los milisegundos (de 0-999).
<code>getMinutes()</code>	Devuelve los minutos (de 0-59).
<code>getMonth()</code>	Devuelve el mes (de 0-11). 0 es enero.
<code>getSeconds()</code>	Devuelve los segundos (de 0-59).
<code>getTime()</code>	Devuelve los milisegundos desde media noche del 1 de enero de 1970.
<code>getTimezoneOffset()</code>	Devuelve la diferencia de tiempo entre GMT y la hora local, en minutos.
<code>setDate()</code>	Ajusta el día del mes del objeto (de 1-31).
<code>setFullYear(año)</code>	Ajusta el año del objeto (4 dígitos).
<code>setHours()</code>	Ajusta la hora del objeto (de 0-23).
<code>setHours(hora)</code>	Ajusta la hora del objeto. Se puede pasar como parámetros separados por comas uno o más según el siguiente formato: hora (0-23), minutos (0-59), segundos (0-59), milisegundos (0-999).
<code>setMilliseconds(ms)</code>	Ajusta los milisegundos del objeto (0-999).
<code>setMinutes(minuto)</code>	Ajusta el minuto del objeto (0-59).
<code>setMonth(mes)</code>	Ajusta el mes del objeto (de 0-11).
<code>setSeconds(segundos)</code>	Ajusta los segundos del objeto (0-59).
<code>setTime(numero)</code>	Ajusta la fecha del objeto, determinada por la cantidad de milisegundos transcurridos desde 1/1/1970.
<code>toString()</code>	Devuelve solamente la fecha.
<code>toISOString()</code>	Devuelve la fecha con formato ISO 8601. Es un estándar donde la fecha y la hora están separadas por el carácter T y finaliza toda la cadena con el carácter Z.
<code>toJSON()</code>	Devuelve la fecha en formato JSON.
<code>toLocaleDateString()</code>	Devuelve la fecha con el formato de la zona configurada en el sistema.

Métodos del objeto Date	
Métodos	Descripción
<code>toLocaleTimeString()</code>	Devuelve la hora con el formato de la zona configurada en el sistema.
<code>toString()</code>	Devuelve solamente la hora.

Propiedades del objeto String	
Propiedad	Descripción
<code>length</code>	Contiene la longitud de una cadena.

Métodos del objeto String	
Métodos	Descripción
<code>codePointAt(pos)</code>	Devuelve el código Unicode del carácter especificado por la posición que se indica como parámetro. El código devuelto estará representado en notación decimal.
<code>concat()</code>	Une una o más cadenas y devuelve el resultado de esa unión. No se utiliza por ser poco legible.
<code>charAt(pos)</code>	Devuelve el carácter de la posición <i>pos</i> . Es equivalente al operador [<i>pos</i>].
<code>endsWith(texto)</code>	Devuelve <i>true</i> si la cadena finaliza con <i>texto</i> .
<code>endsWith(texto, pos)</code>	Devuelve <i>true</i> si la cadena finaliza con <i>texto</i> hasta la posición <i>pos</i> .
<code>fromCharCode()</code>	Convierte valores Unicode a caracteres.
<code>includes(texto)</code>	Devuelve <i>true</i> si la cadena contiene la subcadena <i>texto</i> .
<code>includes(texto, pos)</code>	Devuelve <i>true</i> si la cadena contiene la subcadena <i>texto</i> a partir de la posición <i>pos</i> .
<code>indexOf(texto)</code>	Devuelve la posición de la primera ocurrencia del texto buscado en la cadena. También puedo pasar un segundo parámetro para que busque la ocurrencia del texto a partir desde la posición dada. (<code>indexOf(texto, pos)</code>).
<code>lastIndexOf(texto)</code>	Devuelve la posición de la última ocurrencia del texto buscado en la cadena. También puedo pasar un segundo parámetro para que devuelva la ocurrencia

Métodos del objeto String	
Métodos	Descripción
	del texto desde la posición dada hasta el principio (<code>indexOf(texto, pos)</code>).
<code>match(expreg)</code>	Busca una coincidencia entre una expresión regular y una cadena y devuelve un array con las coincidencias encontradas o <i>null</i> si no ha encontrado, nada. Puede usarse con expresiones regulares.
<code>padEnd(tam, texto)</code>	Devuelve una cadena rellenando el final con <i>texto</i> hasta llegar al tamaño <i>tam</i> .
<code>padStart(tam, texto)</code>	Devuelve una cadena rellenando el inicio con <i>texto</i> hasta llegar al tamaño <i>tam</i> .
<code>repeat(numero)</code>	Devuelve la cadena repetida el número de veces pasado como parámetro.
<code>replace(texto, nuevoTexto)</code>	Busca una subcadena en la cadena y reemplaza la primera aparición por la nueva cadena especificada. Puede usarse con expresiones regulares (<i>texto</i>).
<code>replaceAll(texto, nuevoTexto)</code>	Reemplaza todas las apariciones de <i>texto</i> por <i>nuevoTexto</i> . Puede usarse con expresiones regulares (<i>texto</i>).
<code>search(expreg)</code>	Busca una subcadena en la cadena y devuelve la posición dónde se encontró. Puede usarse con expresiones regulares.
<code>slice(inicio, fin)</code>	Extrae una parte de la cadena y devuelve una nueva cadena. El índice <i>fin</i> queda excluido de la extracción.
<code>split(expreg)</code>	Divide la cadena usando los criterios de la expresión regular <i>expreg</i> .
<code>split(expreg, limite)</code>	Igual que el anterior pero solo devuelve <i>limite</i> fragmentos.
<code>split(texto)</code>	Divide una cadena en un array de subcadenas. Si pasamos un <i>texto</i> como parámetro, separaría la cadena en partes usando <i>texto</i> como separador.
<code>split(texto, limite)</code>	Igual que el anterior pero solo carga <i>limite</i> fragmentos.
<code>startsWith(texto)</code>	Devuelve <i>true</i> si la cadena comienza con <i>texto</i> .
<code>startsWith(texto, pos)</code>	Devuelve <i>true</i> si la cadena comienza con <i>texto</i> desde la posición <i>pos</i> .
<code>String.fromCodePoint(numero)</code>	Devuelve el carácter representado por <i>numero</i> . El número podrá expresarse en notación decimal, hexadecimal,
<code>substr(inicio, tam)</code>	Extrae los caracteres de una cadena, comenzando en una determinada posición y con el número de caracteres indicado.
<code>substring(inicio, fin)</code>	Extrae los caracteres de una cadena entre dos índices especificados. El índice <i>fin</i> queda excluido de la

Métodos del objeto String	
Métodos	Descripción
	extracción.
toLowerCase()	Convierte una cadena en minúsculas.
toUpperCase()	Convierte una cadena en mayúsculas.
trim()	Devuelve una cadena eliminando los espacios en blanco al principio y/o final de una cadena.
trimEnd()	Devuelve una cadena eliminando los espacios a la derecha de la misma.
trimStart()	Devuelve una cadena eliminando los espacios a la izquierda de la misma.